PART-A

PROGRAM 1:

Create a table EMPLOYEE using SQL command to store details of employees such as EMPNO , NAME , DESIGNATION, DEPARTMENT , GENDER and SALARY. Specify Primary Key and NOT NULL constraints on the table Allow only 'M' or 'F' for the column GENDER. DEPARTMENT can be SALES, ACCOUNTS, IT Choose DESIGNATION as CLERK, ANALYST, MANAGER, ACCOUNTANT and SUPERVISOR that depends on department.

Write the following SQL queries:

a) Display EMPNO, NAME and DESIGNATION of all employees whose name ends with RAJ.

b) Display the details of all female employees who is earning salary within the range 20000 to 40000 in SALES or IT departments

c) List the different DEPARTMENTs with the DESIGNATIONs in that department

d) Display the department name, total, average, maximum, minimum salary of the DEPARTMENT only if the total salary given in that department is more than 30000.

e) List the departments which have more than two employees.

Creating table:

SQL> Create table employee (

empno number(10) primary key,

name varchar2(20) not null,

designation varchar2(20) not null,

CHECK (designation IN
('CLERK','ANALYST','MANAGER','ACCOUNTANT','SUPERVISOR')),
department varchar2(20) not null,
CHECK (department IN ('SALES','ACCOUNTS','IT')),
gender char(1) check(gender in('M','F')),
salary number(10,4));

Table created**.**

Inserting values:

SQL> insert into employee values (101, 'arun raj', 'manager', 'sales', 'm', 55000);

1 row created.

SQL> insert into employee values (102, 'priya', 'clerk', 'accounts', 'f', 25000);

1 row created.

SQL> insert into employee values (103, 'manoj raj', 'analyst', 'it', 'm', 42000);

1 row created.

SQL> insert into employee values (104, 'divya', 'accountant', 'accounts', 'f', 32000);

1 row created.

SQL> insert into employee values (105, 'ramesh', 'supervisor', 'sales', 'm', 28000);

1 row created.

SQL> insert into employee values (106, 'jyothi', 'clerk', 'it', 'f', 22000);

1 row created.

SQL> insert into employee values (107, 'arjun', 'analyst', 'it', 'm', 35000);

1 row created.

SQL> insert into employee values (108, 'sandhya', 'supervisor', 'accounts', 'f', 30000);

1 row created.

SQL> insert into employee values (109, 'vinod raj', 'clerk', 'sales', 'm', 20000);

1 row created.

SQL> insert into employee values (110, 'reshma', 'manager', 'it', 'f', 60000);

1 row created.

SQL>select * from employee;

Emp No	Employee Name	Designation	Department	Gender	Salary
101	Arun Raj	Manager	SALES	M	55000
102	Priya	Clerk	ACCOUNTS	F	25000
103	Manoj Raj	Analyst	IT	M	42000
104	Divya	Accountant	ACCOUNTS	F	32000
105	Ramesh	Supervisor	SALES	M	28000
106	Jyothi	Clerk	IT	F	22000
107	Arjun	Analyst	IT	M	35000
108	Sandhya	Supervisor	ACCOUNTS	F	30000
109	Vinod Raj	Clerk	SALES	M	20000
110	Reshma	Manager	IT	F	60000

10 rows selected.

a) Display empno, name and designation of all employees whose name ends with RAJ.

SQL> select empno,name,designation from employee where name like '%Raj';

Emp No	Employee Name	Designation
101	Arun Raj	Manager
103	Manoj Raj	Analyst
109	Vinod Raj	Clerk

b) Display the details of the all females employees who is earning salary within the range of 20000 to 40000 in sales or IT department.

SQL> select * from employee where(gender='F')and(SALARY between 20000 and 40000)and (department in('sales','IT'));

Emp No	Employee Name	Designation	Department	Gender	Salary
106	Jyothi	Clerk	IT	F	22000

c) List the different department with the designation in that department.

SQL>SELECT DISTINCT department FROM employee;

Department
ACCOUNTS
IT
SALES

d) Display the department name,total,average, maximum, minimum salary of the department only if the total salary given in that department is more than 30000.

SQL> select department,
sum(SALARY)"total",avg(salary)"average",max(salary)"maximum",min(salary)"minimum" from

employee group by department having sum(salary)>30000;

Department	total	average	maximum	minimum
ACCOUNTS	87000	29000	32000	25000
IT	159000	39750	60000	22000
SALES	103000	34333.3333	55000	20000

e) List the department which have more than two employees.

SQL>select department from employee group by department having count(*)>2;

Department
ACCOUNTS
IT
SALES

PROGRAM 2:

Create a table CLIENT to store CLIENT_NO, NAME, ADDRESS, STATE, BAL_DUE. Client no must start with 'C'. Apply the suitable structure for the columns. Specify Primary Key and NOT NULL constraints on the table. Insert 10 records.

Write the following SQL queries:

a) From the table CLIENT, create a new table CLIENT1 that contains only CLIENT_NO and NAME, BAL_DUE from specified STATE. Accept the state during run time.

b) Create a new table CLIENT2 that has the same structure as CLIENT but with no records. Display the structure and records.

c) Add a new column by name PENALTY number (10, 2) to table CLIENT.

d) Assign Penalty as 10% of BAL_DUE for the clients C1002, C1005, C1009 and for others 8%. Display records.

e) Change the name of CLIENT1 as NEW_CLIENT.

f) Delete the table client 2.

Creating table:

```
SQL> Create table client (
    client_no varchar2(10) constraint client_pk primary key,
    name varchar2(30) not null,
    address varchar2(50),
    state varchar2(20) not null,
    bal_due number(10,2) not null,
    constraint chk_client_no check (client_no like 'c%')
);
Table created.
```

Inserting values:

```
SQL>insert into client values ('C1001', 'Ravi Kumar', 'Delhi', 'Delhi', 12000.00);
1 row inserted.
SQL>insert into client values ('C1002', 'Anita Sharma', 'Mumbai', 'Maharashtra', 15000.00);
1 row inserted.
SQL>insert into client values ('C1003', 'Vikram Singh','Kolkata','West Bengal',18000.00);
1 row inserted.
```

SQL>insert into client values ('C1004', 'Pooja Nair', 'Chennai', 'Tamil Nadu', 9000.00);

1 row inserted.

SQL>insert into client values ('C1005', 'Rahul Verma', 'Pune', 'Maharashtra', 20000.00);

1 row inserted.

SQL>insert into client values ('C1006', 'Sita Menon', 'Bangalore', 'Karnataka',7500.00);

1 row inserted.

SQL>insert into client values ('C1007', 'Arjun Das', 'Hyderabad', 'Telangana', 11000.00);

1 row inserted.

SQL>insert into client values ('C1008', 'Kiran Rao', 'Manglore', 'Karnataka', 9500.00);

1 row inserted.

SQL>insert into client values ('C1009', 'Meera Gupta', 'Nagpur','Maharashtra',22000.00);

1 row inserted.

SQL>insert into client values ('C1010', 'Sunil Joshi', 'Lucknow', 'Uttar Pradesh',13000.00);

1 row inserted.

SQL> Select ***** form client;

CLIENT_NO	NAME	ADDRESS	STATE	BAL_DUE
C1001	Ravi Kumar	Delhi	Delhi	12000.00
C1002	Anita Sharma	Mumbai	Maharashtra	15000.00
C1003	Vikram Singh	Kolkata	West Bengal	18000.00
C1004	Pooja Nair	Chennai	Tamil Nadu	9000.00
C1005	Rahul Verma	Pune	Maharashtra	20000.00
C1006	Sita Menon	Bangalore	Karnataka	7500.00
C1007	Arjun Das	Hyderabad	Telangana	11000.00
C1008	Kiran Rao	manglore	Karnataka	9500.00
C1009	Meera Gupta	Nagpur	Maharashtra	22000.00
C1010	Sunil Joshi	Lucknow	Uttar Pradesh	13000.00

10 rows selected.

a) From the table CLIENT, create a new table CLIENT1 that contains only CLIENT_NO and NAME, BAL_DUE from specified STATE.Accept the state during run time.

SQL> create table client1 as select client_no, name, bal_due from client where state = '&state';

```
Enter value for state: Karnataka
old   4: where state = '&state'
new   4: where state = 'Karnataka'

Table created.

SQL> select *from client1;

CLIENT_NO    NAME                      BAL_DUE
------------ ------------------------- ----------
C1006        Sita Menon                7500.00
C1008        Kiran Rao                 9500.00
```

b) Create a new table CLIENT2 that has the same structure as CLIENT but with no records. Display the structure and records.

SQL> create table client2 as select * from client where 1=2;

Table created.

SQL> desc client2;

```
Name                                                
```

CLIENT_NO	NAME	ADDRESS	STATE	BAL_DUE	PENALTY
C1001	Ravi Kumar	Delhi	Delhi	12000.00	
C1002	Anita Sharma	Mumbai	Maharashtra	15000.00	
C1003	Vikram Singh	Kolkata	West Bengal	18000.00	
C1004	Pooja Nair	Chennai	Tamil Nadu	9000.00	
C1005	Rahul Verma	Pune	Maharashtra	20000.00	
C1006	Sita Menon	Bangalore	Karnataka	7500.00	
C1007	Arjun Das	Hyderabad	Telangana	11000.00	
C1008	Kiran Rao	manglore	Karnataka	9500.00	
C1009	Meera Gupta	Nagpur	Maharashtra	22000.00	
C1010	Sunil Joshi	Lucknow	Uttar Pradesh	13000.00	

10 rows selected.

d) Assign Penalty as 10% of BAL_DUE for the clients C1002, C1005, C1009 and for others 8%. Display records.

SQL> UPDATE CLIENT SET PENALTY = BAL_DUE * 0.08 WHERE CLIENT_NO NOT IN ('C1002','C1005','C1009');

SQL>UPDATE CLIENT SET PENALTY = BAL_DUE * 0.10 WHERE CLIENT_NO IN ('C1002','C1005','C1009');

SQL> select * from client;

CLIENT_NO	NAME	ADDRESS	STATE	BAL_DUE	PENALTY
C1001	Ravi Kumar	Delhi	Delhi	12000.00	960
C1002	Anita Sharma	Mumbai	Maharashtra	15000.00	1500
C1003	Vikram Singh	Kolkata	West Bengal	18000.00	1440
C1004	Pooja Nair	Chennai	Tamil Nadu	9000.00	720
C1005	Rahul Verma	Pune	Maharashtra	20000.00	2000
C1006	Sita Menon	Bangalore	Karnataka	7500.00	600
C1007	Arjun Das	Hyderabad	Telangana	11000.00	880
C1008	Kiran Rao	manglore	Karnataka	9500.00	760
C1009	Meera Gupta	Nagpur	Maharashtra	22000.00	2200
C1010	Sunil Joshi	Lucknow	Uttar Pradesh	13000.00	1040

10 rows selected.

e) Change the name of CLIENT1 as NEW_CLIENT.

SQL> rename client to new_client;

Table renamed.

SQL> desc new_client;

Name	Null?	Type
CLIENT_NO	NOT NULL	VARCHAR2(10)
NAME	NOT NULL	VARCHAR2(30)
ADDRESS		VARCHAR2(50)
STATE	NOT NULL	VARCHAR2(20)
BAL_DUE	NOT NULL	NUMBER(10,2)
PENALTY		NUMBER(10,2)

SQL> select * from new_client;

CLIENT_NO	NAME	ADDRESS	STATE	BAL_DUE	PENALTY
C1001	Ravi Kumar	Delhi	Delhi	12000.00	960
C1002	Anita Sharma	Mumbai	Maharashtra	15000.00	1500
C1003	Vikram Singh	Kolkata	West Bengal	18000.00	1440
C1004	Pooja Nair	Chennai	Tamil Nadu	9000.00	720
C1005	Rahul Verma	Pune	Maharashtra	20000.00	2000
C1006	Sita Menon	Bangalore	Karnataka	7500.00	600
C1007	Arjun Das	Hyderabad	Telangana	11000.00	880
C1008	Kiran Rao	manglore	Karnataka	9500.00	760
C1009	Meera Gupta	Nagpur	Maharashtra	22000.00	2200
C1010	Sunil Joshi	Lucknow	Uttar Pradesh	13000.00	1040

10 rows selected.

f) Delete the table client 2.

SQL> drop table client2;

Table dropped**.**

PROGRAM 3.

Create a table BOOK using SQL command to store Accession No, TITLE, AUTHOR, PUBLISHER, YEAR, PRICE. Apply the suitable structure for the columns. Specify Primary Key and NOT NULL constraints on the table. Insert 10 records.

Write the following SQL queries:

a) List the details of publishers having 'a' as the second character in their names.

b) Display Accession No., TITLE, PUBLISHER and YEAR of the books published by the specified author before 2010 in the descending order of YEAR. Accept author during run time

c) Modify the size of TITLE to increase the size 5 characters more.

d) Display the details of all books other than Microsoft press publishers.

e) Remove the records of the books published before 1990.

Creating table:

```
SQL> CREATE TABLE book(

accessionno NUMBER(20) PRIMARY KEY,
title VARCHAR2(20) NOT NULL,
author VARCHAR2(20) NOT NULL,
publisher VARCHAR2(20) NOT NULL,
year NUMBER(4) NOT NULL,
price NUMBER(10,2) NOT NULL
);
Table created.
SQL> desc book;
```

Name	Null?	Type
ACCESSIONNO	NOT NULL	NUMBER(20)
TITLE	NOT NULL	VARCHAR2(20)
AUTHOR	NOT NULL	VARCHAR2(20)
PUBLISHER	NOT NULL	VARCHAR2(20)
YEAR	NOT NULL	NUMBER(4)
PRICE	NOT NULL	NUMBER(10,2)

```
SQL> insert into book values (1000, 'Java Fundamentals', 'Herbert Schildt', 'McGraw Hill', 2012,
400.00);
1 row created.
```

SQL> insert into book values (1001, 'programming in c', 'e balaguruswamy', 'epb publications', 2018, 250);

1 row created.

SQL> insert into book values (1002, 'SQL/PL-SQL', 'Ivan Bayross', 'Microsoft Press', 2009, 350);

1 row created.

SQL> insert into book values (1003, 'DBMS', 'Navathe', 'Microsoft Press', 2009, 450);

1 row created.

SQL> insert into book values (1004, 'Fundamentals', 'Elmasri', 'BPB Publications', 1989, 150);

1 row created.

SQL> insert into book values (1005, 'C++', 'Yashavanth', 'Raman Publications', 1987, 150);

1 row created.

SQL> insert into book values (1006, 'C++ Programming', 'Navathe', 'Raman Publications', 2011, 150);

1 row created.

SQL> insert into book values (1007, 'Operating Systems', 'Navathe', 'RS Publications', 2005, 450);

1 row created.

SQL> insert into book values (1008, 'Database Concepts', 'Navathe', 'Pearson Publications', 2008, 300);

1 row created.

SQL> insert into book values (1009, 'System Design', 'Navathe', 'Tata McGraw Hill', 2006, 280);

1 row created.

SQL> select * from book;

ACCESSIONNO	TITLE	AUTHOR	PUBLISHER	YEAR	PRICE
1001	programming in c	e balaguruswamy	epb publications	2018	250
1002	SQL/PL-SQL	Ivan Bayross	Microsoft Press	2009	350
1003	DBMS	Navathe	Microsoft Press	2009	450
1004	Fundamentals	Elmasri	BPB Publications	1989	150
1005	C++	Yashavanth	Raman Publications	1987	150
1006	C++ Programming	Navathe	Raman Publications	2011	150
1007	Operating Systems	Navathe	RS Publications	2005	450
1008	Database Concepts	Navathe	Pearson Publications	2008	300
1009	System Design	Navathe	Tata McGraw Hill	2006	280
1000	Java Fundamentals	Herbert Schildt	McGraw Hill	2012	400

10 rows selected.

a). List the details of publishers having ‘a’ as the second character in their names.

SQL> select * from book where publisher like '_a%';

ACCESSIONNO	TITLE	AUTHOR	PUBLISHER	YEAR	PRICE
1005	C++	Yashavanth	Raman Publications	1987	150
1006	C++ Programming	Navathe	Raman Publications	2011	150
1009	System Design	Navathe	Tata McGraw Hill	2006	280

b). Display Accession No., TITLE, PUBLISHER and YEAR of the books published by the specified author before 2010 in the descending order of YEAR. Accept author during run time.

SQL> select accessionno, title, publisher, year from book where author='Navathe' and year<2010 order by year desc;

ACCESSIONNO	TITLE	PUBLISHER	YEAR
1003	DBMS	Microsoft Press	2009
1008	Database Concepts	Pearson Publications	2008
1009	System Design	Tata McGraw Hill	2006
1007	Operating Systems	RS Publications	2005

c). Modify the size of TITLE to increase the size by 5 characters.

SQL> Alter table book modify title varchar(25);

Table altered.

SQL> DESC book;

Name	Null?	Type
ACCESSIONNO	NOT NULL	NUMBER(20)
TITLE	NOT NULL	VARCHAR2(25)
AUTHOR	NOT NULL	VARCHAR2(20)
PUBLISHER	NOT NULL	VARCHAR2(20)
YEAR	NOT NULL	NUMBER(4)
PRICE	NOT NULL	NUMBER(10,2)

d). Display the details of all books other than Microsoft press publishers.

SQL> select * from book where publisher not in('Microsoft Press');

ACCESSIONNO	TITLE	AUTHOR	PUBLISHER	YEAR	PRICE
1001	programming in c	e balaguruswamy	epb publications	2018	250
1004	Fundamentals	Elmasri	BPB Publications	1989	150
1005	C++	Yashavanth	Raman Publications	1987	150
1006	C++ Programming	Navathe	Raman Publications	2011	150
1007	Operating Systems	Navathe	RS Publications	2005	450
1008	Database Concepts	Navathe	Pearson Publications	2008	300
1009	System Design	Navathe	Tata McGraw Hill	2006	280
1000	Java Fundamentals	Herbert Schildt	McGraw Hill	2012	400

8 rows selected.

e). Remove the records of the books published before 1990.

SQL> delete from book where year<1990;

2 rows deleted.

SQL> select * from book;

ACCESSIONNO	TITLE	AUTHOR	PUBLISHER	YEAR	PRICE
1001	programming in c	e balaguruswamy	epb publications	2018	250
1002	SQL/PL-SQL	Ivan Bayross	Microsoft Press	2009	350
1003	DBMS	Navathe	Microsoft Press	2009	450
1006	C++ Programming	Navathe	Raman Publications	2011	150
1007	Operating Systems	Navathe	RS Publications	2005	450
1008	Database Concepts	Navathe	Pearson Publications	2008	300
1009	System Design	Navathe	Tata McGraw Hill	2006	280
1000	Java Fundamentals	Herbert Schildt	McGraw Hill	2012	400

8 rows selected.

PROGRAM 4:

Create a table SALES with columns SNO, SNAME, MANAGER_NAME, JOIN_DATE, DATE_BIRTH, SALARY, SALES_AMOUNT and COMMISSION. Minimum Age for joining the company must be 18 Yrs. Default value for Commission should be 0. Apply the suitable structure for the columns. Specify Primary Key and NOT NULL constraints on the table. Insert 10 records with data except commission. Manager of Manager can be Null.

Write the following SQL queries:

a) Display the details of Sales Persons whose salary is more than Average salary in the company.

b) Update commission as 20% of Sales Amount.

c) Display SNO, SNAME, MANAGER_NAME, SALARY, COMMISSION, MANAGER_SALARY of the sales persons getting sum of salary and commission more than salary of manager .(Self join)

d) Display the records of employees who finished the service of 10 years.

Creating table:

```
SQL>create table sales
(
sno number(10)
primary key, sname
varchar2(20) not null,
manager_name
varchar2(20),
join_date date not null,
date_birth date not
null, salary
number(10) not null,
```

```
 sales_amount number(10) not null,
 commission number default 0,
 constraint age check(trunc(months_between(join_date, date_birth)/12) >= 18)
);
SQL>insert into sales values (101, 'Lakshmi', 'Suresh', '15-aug-11', '22-jan-86', 2900, 650, default);
1 row inserted.
SQL>insert into sales values (102, 'Suresh', 'Jyothi', '01-jan-10', '01-feb-80', 20000, 50000, default);
1 row inserted.
SQL>insert into sales values (103, 'Suresh', 'Asha', '17-jun-10', '20-oct-80', 30000, 52000, default);
1 row inserted.
SQL>insert into sales values (104, 'Jyothi', 'Raju', '22-jul-99', '15-may-70', 25000, 53000, default);
1 row inserted.
SQL>insert into sales values (105, 'Asha', 'Naveen', '23-jan-09', '24-mar-89', 35000, 55000, default);
1 row inserted.
SQL>insert into sales values (106, 'Raju', 'Akshatha', '02-dec-89', '09-nov-69', 40000, 57000, default);
1 row inserted.
SQL>insert into sales values (107, 'Naveen', 'Madhura', '17-jan-10', '06-oct-79', 42000, 58000,
default);
1 row inserted.
SQL>insert into sales values (108, 'Akshatha', 'Seema', '01-dec-17', '10-apr-87', 45000, 59000,
default);
1 row inserted.
SQL>insert into sales values (109, 'Madhura', 'Theja', '11-may-99', '12-jan-65', 47000, 60000,
default);
1 row inserted.
SQL>insert into sales values (110, 'Seema', 'Rashmi', '30-jun-99', '15-sep-69', 50000, 61000, default);
1 row inserted.
SQL> select *from sales;
```

```
      SNO SNAME                MANAGER_NAME         JOIN_DATE DATE_BIRT  SALARY SALES_AMOUNT COMMISSION
--------- -------------------- -------------------- --------- --------- ------- ------------ ----------
      103 Suresh               Asha                 17-JUN-10 20-OCT-80   30000        52000          0
      101 Lakshmi              Suresh               15-AUG-11 22-JAN-86    2900          650          0
      102 Suresh               Jyothi               01-JAN-10 01-FEB-80   20000        50000          0
      104 Jyothi               Raju                 22-JUL-99 15-MAY-70   25000        53000          0
      105 Asha                 Naveen               23-JAN-09 24-MAR-89   35000        55000          0
      106 Raju                 Akshatha             02-DEC-89 09-NOV-69   40000        57000          0
      107 Naveen               Madhura              17-JAN-10 06-OCT-79   42000        58000          0
      108 Akshatha             Seema                01-DEC-17 10-APR-87   45000        59000          0
      109 Madhura              Theja                11-MAY-99 12-JAN-65   47000        60000          0
      110 Seema                Rashmi               30-JUN-99 15-SEP-69   50000        61000          0

10 rows selected.
```

a) Display the details of sales persons whose salary is more than average salary in the company.

SQL> select * from sales where salary>(select avg(salary) from sales);

```
SQL> select * from sales where salary>(select avg(salary) from sales);

      SNO SNAME                MANAGER_NAME         JOIN_DATE DATE_BIRT  SALARY SALES_AMOUNT COMMISSION
--------- -------------------- -------------------- --------- --------- ------- ------------ ----------
      105 Asha                 Naveen               23-JAN-09 24-MAR-89   35000        55000          0
      106 Raju                 Akshatha             02-DEC-89 09-NOV-69   40000        57000          0
      107 Naveen               Madhura              17-JAN-10 06-OCT-79   42000        58000          0
      108 Akshatha             Seema                01-DEC-17 10-APR-87   45000        59000          0
      109 Madhura              Theja                11-MAY-99 12-JAN-65   47000        60000          0
      110 Seema                Rashmi               30-JUN-99 15-SEP-69   50000        61000          0

6 rows selected.
```

b) Update commission as 20% of Sales Amount.

SQL> update sales set commission = (sales_amount * 0.20);

10 rows updated.

```
SQL> select *from sales;

      SNO SNAME                MANAGER_NAME         JOIN_DATE DATE_BIRT  SALARY SALES_AMOUNT COMMISSION
--------- -------------------- -------------------- --------- --------- ------- ------------ ----------
      103 Suresh               Asha                 17-JUN-10 20-OCT-80   30000        52000      10400
      101 Lakshmi              Suresh               15-AUG-11 22-JAN-86    2900          650        130
      102 Suresh               Jyothi               01-JAN-10 01-FEB-80   20000        50000      10000
      104 Jyothi               Raju                 22-JUL-99 15-MAY-70   25000        53000      10600
      105 Asha                 Naveen               23-JAN-09 24-MAR-89   35000        55000      11000
      106 Raju                 Akshatha             02-DEC-89 09-NOV-69   40000        57000      11400
      107 Naveen               Madhura              17-JAN-10 06-OCT-79   42000        58000      11600
      108 Akshatha             Seema                01-DEC-17 10-APR-87   45000        59000      11800
      109 Madhura              Theja                11-MAY-99 12-JAN-65   47000        60000      12000
      110 Seema                Rashmi               30-JUN-99 15-SEP-69   50000        61000      12200

10 rows selected.
```

c). Display SNO, SNAME, MANAGER_NAME, SALARY, COMMISSION, MANAGER_SALARY of the sales persons getting sum of salary and commission more than salary of manager .(Self join)

SQL> select s.sno, s.sname, s.manager_name, s.salary, s.commission, m.salary as manager_salary from sales s join sales m on s.manager_name = m.sname where (s.salary + s.commission) > m.salary;

SNO	SNAME	MANAGER_NAME	SALARY	COMMISSION	MANAGER_SALARY
102	Suresh	Jyothi	20000	10000	25000
103	Suresh	Asha	30000	10400	35000
105	Asha	Naveen	35000	11000	42000
106	Raju	Akshatha	40000	11400	45000
107	Naveen	Madhura	42000	11600	47000
108	Akshatha	Seema	45000	11800	50000

6 rows selected.

d)Display the records of employees who finished the service of 10 years.

SQL> Select *from sales where trunc(months_between(sysdate,join_date))>10;

SNO	SNAME	MANAGER_NAME	JOIN_DATE	DATE_BIRT	SALARY	SALES_AMOUNT	COMMISSION
103	Suresh	Asha	17-JUN-10	20-OCT-80	30000	52000	10400
101	Lakshmi	Suresh	15-AUG-11	22-JAN-86	2900	650	130
102	Suresh	Jyothi	01-JAN-10	01-FEB-80	20000	50000	10000
104	Jyothi	Raju	22-JUL-99	15-MAY-70	25000	53000	10600
105	Asha	Naveen	23-JAN-09	24-MAR-89	35000	55000	11000
106	Raju	Akshatha	02-DEC-89	09-NOV-69	40000	57000	11400
107	Naveen	Madhura	17-JAN-10	06-OCT-79	42000	58000	11600
108	Akshatha	Seema	01-DEC-17	10-APR-87	45000	59000	11800
109	Madhura	Theja	11-MAY-99	12-JAN-65	47000	60000	12000
110	Seema	Rashmi	30-JUN-99	15-SEP-69	50000	61000	12200

10 rows selected.

PROGRAM 5.

Create a table Sales_Details with the columns SNO, MONTH, TARGET and QTY_SOLD to store the Sales Details of one year. Specify the composite primary key to the columns SNO and MONTH. TARGET and SALES must be positive numbers.

Write the following SQL queries:

a) Display the total sales by each sales person considering only those months sales where target was reached

b) If a commission of RS.50 provided for each item after reaching target, calculate and display the total commission for each sales person.

c) Display the SNO of those who never reached the target. d) Display the SNO, MONTH and QTY_SOLD of the sales persons with SNO S0001 or S0003

Creating table:

```
SQL> create table sales_details (
 sno  varchar2(10),
 month varchar2(10),
 target number(10) check (target > 0),
 qty_sold  number(10) check (qty_sold > 0),
 primary key (sno, month)
 );
Table created.
SQL> INSERT INTO sales_details VALUES('s0001','dec',60,60);
1 row created.
SQL> INSERT INTO sales_details VALUES('s0002','feb',50,70);
1 row created.
SQL> INSERT INTO sales_details VALUES('s0003','aug',60,50);
1 row created.
SQL> INSERT INTO sales_details VALUES('s0004','oct',40,23);
1 row created.
```

SQL> INSERT INTO sales_details VALUES('s0005','nov',100,120);

1 row created.

SQL> INSERT INTO sales_details VALUES('s0006','jan',80,90);

1 row created.

SQL> select *from sales_details;

SNO	MONTH	TARGET	QTY_SOLD
s0001	dec	60	60
s0002	feb	50	70
s0003	aug	60	50
s0004	oct	40	23
s0005	nov	100	120
s0006	jan	80	90

6 rows selected.

Write the following SQL queries:

a) Display the total sales by each sales person considering only those months sales where target was reached .

SQL> select sno, sum(qty_sold) as total_sales from sales_details
where qty_sold >= target group by sno;

SNO	TOTAL_SALES
s0002	70
s0001	60
s0005	120
s0006	90

b) If a commission of Rs.50 provided for each item after reaching target, calculate and display the total commission for each sales person

SQL> select sno, sum(qty_sold*50) as "COMMISSION_EARNED"
from sales_details where qty_sold >= target
group by sno;

SNO	COMMISSION_EARNED
s0002	3500
s0001	3000
s0005	6000
s0006	4500

c) Display the SNO of those who never reached the target.

SQL> select sno from sales_details where sno not in (select sno from sales_details where target<=qty_sold);

SNO
s0003
s0004

d) Display the SNO, MONTH, and QTY_SOLD of the sales person with SNO S0001 or S0003.

SQL> select sno, month, qty_sold from sales_details where sno='s0001' or sno='s0003';

SNO	MONTH	QTY_SOLD
s0001	dec	60
s0003	aug	50

PROGRAM 6.

Create the following tables by identifying primary and foreign keys. Specify the not null property for mandatory keys.

SUPPLIERS(SUPPLIER_NO,SNAME,SADDRESS,SCITY)

COMPUTER_ITEMS(ITEM_NO,SUPPLIER_NO,ITEM_NAME, IQUANTITY)

Consider three suppliers. A supplier can supply more than one type of items.

Write the SQL queries for the following:

a) List ITEM and SUPPLIER details in alphabetical order of city name and in each city decreasing order of IQUANTITY.

b) List the name ,city,and address of the suppliers who are supplying keyboard.

c) List the supplier name, items supplied by the suppliers 'Cats' and 'Electrotech'.

d) Find the items having quantity less than 5 and insert the details of supplier and item of these, into another table NEWORDER

Creating table:

```
SQL>Create table suppliers(
sno varchar(10) primary key,
sname varchar(20) not null,
saddr varchar(15),
scity varchar(15));
Table created.
```

Inserting Values:

```
SQL>insert into suppliers values('s01','microtech','lalbagh','bangalore');
1 row created.
SQL>insert into suppliers values('s02','cats','bombay','mumbai');
1 row created.
SQL>insert into suppliers values('s03','electrotech','madras','chennai');
1 row created.
SQL>insert into suppliers values('s04','mice','knagar','kodagu');
1 row created.
SQL>insert into suppliers values('s05','electrotech','udupi','mangalore');
1 row created.
```

SQL> select * from suppliers;

SNO	SNAME	SADDR	SCITY
s01	microtech	lalbagh	bangalore
s02	cats	bombay	mumbai
s03	electrotech	madras	chennai
s04	mice	knagar	kodagu
s05	electrotech	udupi	mangalore

Creating table:

SQL> Create table citems(

item_no varchar(10) primary

key,

supplier_no varchar(10) references

suppliers(sno), item_name varchar(10) not

null,

iquantity number(4)

);

Table created.

Inserting values:

SQL> insert into citems values('101','s01','keyboard',4);

1 row created.

SQL> insert into citems values('102','s02','mouse',5);

1 row created.

SQL> insert into citems values('103','s03','printer',10);

1 row created.

SQL> insert into citems values('104','s04','mouse',20);

1 row created.

SQL> insert into citems values('105','s05','keyboard',10);

1 row created.

SQL> select *from citems;

ITEM_NO	SUPPLIER_N	ITEM_NAME	IQUANTITY
101	s01	keyboard	4
102	s02	mouse	5
103	s03	printer	10
104	s04	mouse	20
105	s05	keyboard	10

a) List item and supplier details in alphabetical order of city name and in each city decreasing order of iquantity.

SQL> select i.item_no, i.item_name, s.sno as supplier_no, s.sname, s.scity, i.iquantity from citems i join suppliers s on i.supplier_no = s.sno order by s.scity asc, i.iquantity desc;

ITEM_NO	ITEM_NAME	SUPPLIER_N	SNAME	SCITY	IQUANTITY
101	keyboard	s01	microtech	bangalore	4
103	printer	s03	electrotech	chennai	10
104	mouse	s04	mice	kodagu	20
105	keyboard	s05	electrotech	mangalore	10
102	mouse	s02	cats	mumbai	5

b) List the name ,city,and address of the suppliers who are supplying keyboard

SQL> select sname, scity, saddr from suppliers where sno in (select supplier_no from citems where item_name = 'keyboard');

SNAME	SCITY	SADDR
microtech	bangalore	lalbagh
electrotech	mangalore	udupi

c) List the supplier name, items supplied by the suppliers ‘cats’ and ‘electrotech’.

SQL> select s.sname, c.item_name from suppliers s, citems c where s.sno = c.supplier_no and s.sname in ('cats', 'electrotech');

SNAME	ITEM_NAME
cats	mouse
electrotech	printer
electrotech	keyboard

d) Find the items having quantity less than 5 and insert the details of supplier and item of these, into another table NEWORDER

SQL> Create table neworder as select s.sno, s.sname, s.saddr, s.scity, i.item_no, i.item_name, i.iquantity from suppliers s, citems i where s.sno = i.supplier_no and i.iquantity < 5;

SQL>select *from neworder;

SNO	SNAME	SADDR	SCITY	ITEM_NO	ITEM_NAME	IQUANTITY
S01	microtech	lalbagh	bangalore	101	keyboard	4

PROGRAM 7.

Create the following tables by identifying primary and foreign keys, specify the not null property for mandatory keys.

PRODUCT_DETAIL				
P_NO	**PRODUCTNAME**	**QTYAVAILABLE**	**PRICE**	**PROFIT %**
P0001	Monitor	10	3000	20
P0002	Pen Drives	50	650	5
P0003	CD Drive	100	10	3
P0004	Key Board	25	600	10

PURCHASED_DETAIL		
CUSTNO	**P_NO**	**QTYSOLD**
C1	P0003	2
C2	P0002	4
C3	P0002	10
C4	P0001	3
C1	P0004	2
C2	P0003	2
C4	P0004	1

Write the following SQL queries:

a) Display total amount spent by C2.

b) Display the names of product for which either QtyAvailable is less than 30 or total QtySold is less than 5(USE UNION).

c) Display the name of products and quantity purchased by C4.

d) How much Profit does the shopkeeper gets on C1's purchase?

e) How many 'Pen Drives' have been sold?

Creating tables:

```
SQL>Create table product_detail (
   p_no varchar2(10) primary key,
```

```
    productname varchar2(50) not null,
    qtyavailable number not null,
    price number not null,
    profit number
);
```

Table created.

Inserting values:

```
SQL>insert into product_detail values ('P0001', 'Monitor', 10, 3000, 20);
1 row created.
SQL> insert into product_detail values ('P0002', 'Pen Drives', 50, 650, 5);
1 row created.
SQL> insert into product_detail values ('P0003', 'CD Drive', 100, 10, 3);
1 row created.
SQL> insert into product_detail values ('P0004', 'Key Board', 25, 600, 10);
1 row created.
SQL> Select * from product_detail;
```

P_NO	PRODUCTNAME	QTYAVAILABLE	PRICE	PROFIT
P0001	Monitor	10	3000	20
P0002	Pen Drives	50	650	5
P0003	CD Drives	100	10	3
P0004	Keyboard	25	600	10

Creating table:

```
SQL> Create table purchased_detail (
custno varchar2(10) not null,
p_no varchar2(10) not null,
qtysold number not null,
constraint fk_pno foreign key (p_no) references product_detail(p_no)
);
```

Table created.

Inserting values:

SQL> insert into purchased_detail values ('C1', 'P0003', 2);

1 row created.

SQL> insert into purchased_detail values ('C2', 'P0002', 4);

1 row created.

SQL> insert into purchased_detail values ('C3', 'P0002', 10);

1 row created.

SQL> Insert into purchased_detail values ('C4', 'P0001', 3);

1 row created.

SQL> Insert into purchased_detail values ('C1', 'P0004', 2);

1 row created.

SQL> Insert into purchased_detail values ('C2', 'P0003', 2);

1 row created.

SQL> Insert into purchased_detail values ('C4', 'P0004', 1);

1 row created.

SQL> Select * from purchased_detail;

```
CUSTNO     P_NO          QTYSOLD
---------- ---------- ----------
C1         P0003               2
C2         P0002               4
C3         P0002              10
C4         P0001               3
C1         P0004               2
C2         P0003               2
C4         P0004               1

7 rows selected.
```

a) **Display total amount spent by C2.**

SQL>select pd.custno, sum(p.price * pd.qtysold) as total_amount_spent from purchased_detail pd join product_detail p on pd.p_no = p.p_no where pd.custno = 'c2' group by pd.custno;

```
CU TOTAL_AMOUNT_SPENT
-- ------------------
C2                2620
```

b) **Display the names of product for which either QtyAvailable is less than 30 or total QtySold is less than 5 (Use UNION).**

SQL>select productname from product_detail where qtyavailable < 30 union select p.productname from product_detail p join purchased_detail pd on p.p_no = pd.p_no group by p.productname having sum(pd.qtysold) < 5;

```
PRODUCTNAME
---------------
Monitor
Key Board
CD Drive
```

c) Display the name of products and quantity purchased by C4.

SQL> select p.productname, pd.qtysold from purchased_detail pd join product_detail p on pd.p_no = p.p_no where pd.custno = 'c4';

```
PRODUCTNAME                                        QTYSOLD
-------------------------------------------------- ----------
Monitor                                                 3
Key Board                                               1
```

d) How much Profit does the shopkeeper get on C1's purchase?

SQL> select pd.custno, sum((p.price * pd.qtysold * p.profit) / 100) as total_profit from purchased_detail pd join product_detail p on pd.p_no = p.p_no where pd.custno = 'c1' group by pd.custno;

```
CU TOTAL_PROFIT
-- ------------
C1         120.6
```

e) **How many 'Pen Drives' have been sold?**

SQL> select sum(pd.qtysold) as total_pendrives_sold from purchased_detail pd join product_detail p on pd.p_no = p.p_no where p.productname = 'pen drives';

```
TOTAL_PENDRIVES_SOLD
--------------------
                  14
```

PART B

PROGRAM 1.

Create a table Bank with the columns ACNO, ACT_NAME, ACT_TYPE and BAL. Specify the Primary Key. Initial BAL must be greater than 500.

Write a PL/SQL program to perform debit operation by providing acct_no and amount required. The amount must be greater than 100 and less than 20000 for one transaction. If the account exist and BAL-amount>500 Bank table must be updated, otherwise "NO SUFFFICIENT BALANCE" message should be displayed. If account number is not present then display "NO SUCH ACCOUNT" message to the user.

Creating table:

```
SQL> Create table bank (
acno number(10) primary key,
act_name varchar2(30),
act_type varchar2(10),
bal number(10,2) check (bal > 500)
 );
Table created.
```

Inserting values:

```
SQL>Insert into bank values (101, 'ravi', 'saving', 2000);

1 row created.

SQL>Insert into bank values (102, 'meena', 'current', 1500);
1 row created.
SQL>Insert into bank values (103, 'john', 'saving', 600);
1 row created.
SQL> Set Serveroutput on;
```

PL/SQL code:

```
Declare

   v_acno bank.acno%type := &enter_account_no;
   v_amount number := &enter_amount;
```

```
    v_balance bank.bal%type;
Begin
    select bal into v_balance
    from bank
    where acno = v_acno;
if v_amount < 100 or v_amount > 20000 then
        dbms_output.put_line('amount must be between 100 and 20000.');
elsif (v_balance - v_amount) > 500 then
        update bank
        set bal = bal - v_amount
        where acno = v_acno;
        dbms_output.put_line('amount of rs. ' || v_amount || ' debited successfully.');
        dbms_output.put_line('remaining balance: ' || (v_balance - v_amount));
       else
        dbms_output.put_line('no sufficient balance.');
    End if;
Exception
    when no_data_found then
        dbms_output.put_line('no such account.');
End;
/
SQL> @ "C:\Users\Desktop\Bank.sql"
```

```
Enter value for enter_account_no: 101
old   2:     v_acno bank.acno%type := &enter_account_no;
new   2:     v_acno bank.acno%type := 101;
Enter value for enter_amount: 2500
old   3:     v_amount number := &enter_amount;
new   3:     v_amount number := 2500;
no sufficient balance.

PL/SQL procedure successfully completed.
```

SQL> @ "C:\Users\Desktop\Bank.sql"

```
Enter value for enter_account_no: 102
old   2:     v_acno bank.acno%type := &enter_account_no;
new   2:     v_acno bank.acno%type := 102;
Enter value for enter_amount: 500
old   3:     v_amount number := &enter_amount;
new   3:     v_amount number := 500;
amount of rs. 500 debited successfully.
remaining balance: 1000

PL/SQL procedure successfully completed.
```

SQL> @ "C:\Users\Desktop\Bank.sql"

```
Enter value for enter_account_no: 104
old   2:     v_acno bank.acno%type := &enter_account_no;
new   2:     v_acno bank.acno%type := 104;
Enter value for enter_amount: 500
old   3:     v_amount number := &enter_amount;
new   3:     v_amount number := 500;
no such account.

PL/SQL procedure successfully completed.
```

PROGRAM2.

Create a table STOCK_DETAIL with the columns PNO, PNAME and QTY_AVL to store stock details of computer accessories. Specify Primary Key and NOT NULL constraints on the table. QTY_AVL should be positive number.

Write a PL/SQL Program to define a user defined exception named "LOW_STOCK" to validate the transaction. The program facilitates the user to purchase the product by providing product number and quantity required. It should display an error message "NO SUFFICIENT STOCK" when the user tries to purchase a product with quantity more than QTY_AVL, Otherwise the STOCK_DETAIL table should be updated for valid transaction.

Creating table:

```
SQL> Create table stock_detail (
pno varchar2(10) primary key,
pname varchar2(30) not null,
qty_avl number(5) check (qty_avl > 0)
);
Table created.
```

Inserting values:

```
SQL> Insert into stock_detail values ('p001', 'keyboard', 20);
1 row created.

SQL> Insert into stock_detail values ('p002', 'mouse', 15);
1 row created.

SQL> Insert into stock_detail values ('p003', 'monitor', 10);
1 row created.

SQL> Insert into stock_detail values ('p004', 'usb cable', 30);
1 row created.
```

SQL> Set Serveroutput on;

PL/SQL code:

```
declare
    v_pno stock_detail.pno%type := '&enter_product_no';
    v_qty_req number := &enter_quantity;
    v_qty_avl stock_detail.qty_avl%type;
low_stock exception;
begin
    select qty_avl into v_qty_avl
    from stock_detail
    where pno = v_pno;
if v_qty_req > v_qty_avl then
        raise low_stock;
else
update stock_detail
set qty_avl = qty_avl - v_qty_req where
pno = v_pno;
dbms_output.put_line('purchase successful!'); dbms_output.put_line('remaining stock:
' || (v_qty_avl - v_qty_req));
    end if;
exception
    when no_data_found then
        dbms_output.put_line('no such product found.');
    when low_stock then
        dbms_output.put_line('no sufficient stock.');
end;
/
```

SQL> @ "C:\Users\Desktop\StockDetails.sql"

```
Enter value for enter_product_no: p002
old   2:     v_pno stock_detail5.pno%type := '&enter_product_no';
new   2:     v_pno stock_detail5.pno%type := 'p002';
Enter value for enter_quantity: 5
old   3:     v_qty_req number := &enter_quantity;
new   3:     v_qty_req number := 5;
purchase successful!
remaining stock: 10

PL/SQL procedure successfully completed.
```

SQL> @ "C:\Users\Desktop\StockDetails.sql"

```
Enter value for enter_product_no: p002
old   2:     v_pno stock_detail5.pno%type := '&enter_product_no';
new   2:     v_pno stock_detail5.pno%type := 'p002';
Enter value for enter_quantity: 20
old   3:     v_qty_req number := &enter_quantity;
new   3:     v_qty_req number := 20;
no sufficient stock.

PL/SQL procedure successfully completed.
```

SQL> @ "C:\Users\Desktop\StockDetails.sql"

```
Enter value for enter_product_no: p006
old   2:     v_pno stock_detail5.pno%type := '&enter_product_no';
new   2:     v_pno stock_detail5.pno%type := 'p006';
Enter value for enter_quantity: 8
old   3:     v_qty_req number := &enter_quantity;
new   3:     v_qty_req number := 8;
no such product found.

PL/SQL procedure successfully completed.
```

PROGRAM 3.

Write a PL/SQL program to compute the selling price of books depending on the book code and category. Use Open, Fetch and Close. The Book_detail table contains columns: Book Code, Author, Title, Category and Price. Insert 10 records. The selling price=Price-Discount.

The discount is calculated as follows:

Book Code	Category	Discount Percentage
A	Novels	10% of Price
	Technology	12.5% of Price
B	Commerce	18% of Price
	Science	19% of Price
C	Songs	25% of Price
	Sports	24% of Price
D	All	28% of Price

Print the result in tabular form with proper alignment

```
Book Code   category   title    author   price  discount %  discount amount  sell price
========   =====    ===  =====  ===   =======   =====  =====  =====
```

Creating table:

```
SQL> Create table book (
book_code varchar2(10),
author varchar2(50),
title varchar2(100),
category varchar2(20),
price number(8, 2)
);
Table created.
```

Inserting values:

```
SQL> insert into book values ('A', 'rashmi', 'infotechnology', 'novels', 125);
1 row created.
SQL> insert into book values ('A', 'likith', 'microtechnology', 'technology', 900);
1 row created.
 SQL> insert into book values ('B', 'gagan', 'ecommerce', 'commerce', 1500);
 1 row created.
```

SQL> insert into book values ('B', 'vijeth', 'relatives', 'science', 2500);

1 row created.

SQL> insert into book values ('C', 'jerry', 'go ask', 'song', 500);

1 row created.

SQL> insert into book values ('C', 'tom', 'mind', 'sports', 400);

1 row created.

SQL> insert into book values ('d', 'henry', 'mask', 'others', 3000);

1 row created.

SQL> Select * from book;

BOOK_CODE	AUTHOR	TITLE	CATEGORY	PRICE
A	rashmi	infotechnology	novels	125
A	likith	microtechnology	technology	900
B	gagan	ecommerce	commerce	1500
B	vijeth	relatives	science	2500
C	jerry	go ask	song	500
C	tom	mind	sports	400
D	henry	mask	others	3000

7 rows selected.

SQL>set serveroutput on;

PL/SQL code:

```
declare
    cursor book_cur is
   select book_code, author, title, category, price from book;
v_code book.book_code%type;
v_auth book.author%type;
v_title book.title%type;
v_cat book.category%type;
v_price book.price%type;
v_disc number(5,2);
v_sell number(8,2);
```

```
 begin
  dbms_output.put_line('----------------------------------------');
  dbms_output.put_line('code category title author price disc% sell');
  dbms_output.put_line('----------------------------------------');
open book_cur;
loop
  fetch book_cur into v_code, v_auth, v_title, v_cat, v_price;
  exit when book_cur%notfound;
if (v_code='A' and v_cat='novels') then
  v_disc := 10;
elsif (v_code='A' and v_cat='technology') then
  v_disc := 12.5;
elsif (v_code='B' and v_cat='commerce') then
  v_disc := 18;
elsif (v_code='B' and v_cat='science') then
  v_disc := 19;
elsif (v_code='C' and v_cat='song') then
  v_disc := 25;
elsif (v_code='C' and v_cat='sports') then
  v_disc := 24;
elsif (v_code='D' and v_cat='others') then
  v_disc := 28;
else
 v_disc := 0;
end if;
  v_sell := v_price - (v_price * v_disc / 100);
  dbms_output.put_line(v_code || ' ' || v_cat || ' ' || v_title || ' ' || v_auth || ' ' || v_price || ' ' || v_disc || '% ' ||
v_sell);
  end loop;
  close book_cur;
```

```
  end;
/
```

SQL> @ "C:\Users\Desktop\Book.sql"

CODE	CATEGORY	TITLE	AUTHOR	PRICE	DISC%	SELL
A	NOVELS	INFOTECHNOLOGY	RASHMI	125	10%	112.5
A	TECHNOLOGY	MICROTECHNOLOGY	LIKITH	900	12.5%	787.5
B	COMMERCE	ECOMMERCE	GAGAN	1500	18%	1230
B	SCIENCE	RELATIVES	VIJETH	2500	19%	2025
C	SONG	GO ASK	JERRY	500	25%	375
C	SPORTS	MIND	TOM	400	24%	304
D	OTHERS	MASK	HENRY	3000	28%	2160

PROGRAM 4.

Write a PL/SQL program to display employee pay bill (using Cursor For loop) Use a Procedure to receive basic pay and to compute DA, HRA, Tax, PF, Gross Pay and Net Pay(Use OUT). Base table contains the following columns empnum, empname, basic pay. Insert 3 records.

Basic Pay	DA	HRA
<=20000	35% of Basic	8% of Basic
>20000 & <=30000	38%	9%
>30000 & <=40000	40%	10%
>40000	45%	10%

Allowances are computed as follows.

Gross=Basic+DA+HRA

PF=12% of Gross or Rs. 2000 whichever is minimum.

PT=Rs. 100 upto Gross is 25,000 else Rs. 200. Net=Gross-(PF+PT)

Print Pay slip as follows

```
==========PAYSLIP==========
Empno          :10011          Empname : Raj
Basic Pay      :20000          P.F.: 3432
DA             :7000           P.T.: 200
H.R.A.         :1600
Gross          :28600          Net Pay : 24968
**********************************************
==========PAYSLIP==========
Empno          :10012          Empname : Rani
Basic Pay      :30000          P.F.: 5292
DA             :11400          P.T.: 200
H.R.A.         :2700
Gross          :44100          Net Pay : 38608
**********************************************
```

Creating table:

```
SQL> create table employee (
empno varchar2(14) primary key,
ename varchar2(20) not null,
basic number(10,2) not null
);
```

Table created.

Inserting values:

```
SQL> insert into employee values(1001,'radha',20000);
```

1 row created.

SQL> insert into employee values(1002,'vishwa',30000);

1 row created.

SQL> insert into employee values(1003,'krish',5000);

1 row created.

SQL> insert into employee values(1004,'naveen',45000);

1 row created.

SQL> insert into employee values(1005,'munna',5000);

1 row created.

SQL> select *from employee;

EMPNO	ENAME	BASIC
1001	radha	20000
1002	vishwa	30000
1003	krish	5000
1004	naveen	45000
1005	munna	5000

SQL>set serveroutput on;

PL/SQL code:

```
CREATE OR REPLACE PROCEDURE  paybill
(
    p_basic IN NUMBER,
    p_da OUT NUMBER,
    p_hra OUT NUMBER,
    p_tax OUT NUMBER,
    p_pf OUT NUMBER,
    p_gross OUT NUMBER,
    p_net OUT NUMBER
```

```
)
IS
BEGIN
   IF p_basic <= 20000 THEN
      p_da := p_basic * 0.35;
      p_hra := p_basic * 0.80;
   ELSIF p_basic > 20000 AND p_basic <= 30000 THEN
      p_da := p_basic * 0.38;
      p_hra := p_basic * 0.90;
   ELSIF p_basic > 30000 AND p_basic <= 40000 THEN
      p_da := p_basic * 0.40;
      p_hra := p_basic * 0.10;
   ELSE
      p_da := p_basic * 0.45;
      p_hra := p_basic * 0.10;
   END IF;
   p_gross := p_basic + p_da + p_hra;
   p_pf := p_gross * 0.12;
   IF p_pf >= 2000 THEN
      p_pf := 2000;
   END IF;
   IF p_gross <= 25000 THEN
```

```
        p_tax := 100;
    ELSE
        p_tax := 200;
    END IF;
    p_net := p_gross - (p_pf + p_tax);
END;
/
DECLARE
    v_da NUMBER(10,2);
    v_hra NUMBER(10,2);
    v_tax NUMBER(10,2);
    v_pf NUMBER(10,2);
    v_gross NUMBER(10,2);
    v_net NUMBER(10,2);
BEGIN
    FOR emp_rec IN
        (SELECT empno, ename, basic
         FROM employee)
    LOOP
        paybill( emp_rec.basic, v_da,  v_hra, v_tax, v_pf, v_gross,  v_net);
        DBMS_OUTPUT.PUT_LINE('================================');
        DBMS_OUTPUT.PUT_LINE('         EMPLOYEE PAYSLIP       ');
```

```
        DBMS_OUTPUT.PUT_LINE('================================');
        DBMS_OUTPUT.PUT_LINE('Emp No     : ' || emp_rec.empno);
        DBMS_OUTPUT.PUT_LINE('Emp Name   : ' || emp_rec.ename);
        DBMS_OUTPUT.PUT_LINE('Basic Pay  : ' || emp_rec.basic);
        DBMS_OUTPUT.PUT_LINE('DA         : ' || v_da);
        DBMS_OUTPUT.PUT_LINE('HRA        : ' || v_hra);
        DBMS_OUTPUT.PUT_LINE('Gross Pay  : ' || v_gross);
        DBMS_OUTPUT.PUT_LINE('PF         : ' || v_pf);
        DBMS_OUTPUT.PUT_LINE('Tax        : ' || v_tax);
        DBMS_OUTPUT.PUT_LINE('Net Pay    : ' || v_net);
        DBMS_OUTPUT.PUT_LINE('================================');
    END LOOP;
END;
/
```

SQL> @ "C:\Users\Desktop\Employee.sql"

```
Emp no :1001 Emp name :radha
Basicpay :20000 PF :2000
DA :7000 PT :200
H.R.A :16000
Gross :43000
**********************************
=============PAYSLIP============
Emp no :1002 Emp name :vishwa
Basicpay :30000 PF :2000
DA :11400 PT :200
H.R.A :27000
Gross :68400
**********************************
=============PAYSLIP============
Emp no :1003 Emp name :krish
Basicpay :5000 PF :1290
DA :1750 PT :100
H.R.A :4000
Gross :10750
**********************************
=============PAYSLIP============
Emp no :1004 Emp name :naveen
Basicpay :45000 PF :2000
DA :20250 PT :200
H.R.A :4500
Gross :69750
**********************************
=============PAYSLIP============
Emp no :1005 Emp name :munna
Basicpay :5000 PF :1290
DA :1750 PT :100
H.R.A :4000
Gross :10750
**********************************

PL/SQL procedure successfully completed.
```

PROGRAM 5.

Given the following tables: ITEM_MASTER(itemno, name, stock, unit_price) [Apply the Primary key and check constraint for stock and price as >0] [Insert 5 records] ITEM_TRANS (itemno, quantity and trans_date)

Create a package PCK_ITEM includes a function CHK_ITEM and a procedure PROC_ITEM. Function CHK_ITEM gets one argument itemno and is used to check whether the parameter itemno exists in ITEM_MASTER and should return 1 if exist. Otherwise 0 and displays proper message.

Procedure PROC_ITEM gets two arguments itemno and quantity, and is used to perform the following if item exists. If required quantity is not available, give appropriate message. If available , insert a record of this transaction to ITEM_TRANS and modify the stock in ITEM_MASTER.

Write a PL/SQL program to accept ITEM_NO and Quantity needed of required item. Use Package to do the transaction process (Transaction date can be current date). OUTPUT to be shown as follows:

```
Enter value for accept_itemno: 1
old   5:            X:=&accept_itemno;
new   5:            X:=1;
Enter value for quantity: 3
old   6:          M:=&quantity;
new   6:          M:=3;
Item :aa   Quantity  :3  Price  :15 Total Amount  :45
```

Creating tables:

```
SQL> CREATE TABLE item_master
 (
    itemno NUMBER PRIMARY KEY,
    name VARCHAR2(20),
    stock NUMBER CHECK (stock > 0),
    unit_price NUMBER CHECK (unit_price > 0)
  );
Table created.
```

```
SQL> CREATE TABLE item_trans
  (
    itemno NUMBER,
    quantity NUMBER,
    trans_date DATE
  );
Table created.
```

Inserting values:

```
SQL> INSERT INTO item_master VALUES (1, 'Mouse', 10, 15);
1 row created.
SQL> INSERT INTO item_master VALUES (2, 'Keyboard', 15, 20);
1 row created.
SQL> INSERT INTO item_master VALUES (3, 'CPU', 10, 20);
1 row created.
SQL> INSERT INTO item_master VALUES (4, 'Monitor', 5, 20);
1 row created.
SQL> INSERT INTO item_master VALUES (5, 'Harddisk', 17, 20);
1 row created.
SQL> COMMIT;
Commit complete.
```

```
SQL> SELECT * FROM item_master;
```

ITEMNO	NAME	STOCK	UNIT_PRICE
1	Mouse	10	15
2	Keyboard	15	20
3	CPU	10	20
4	Monitor	5	20
5	Harddisk	17	20

Creating Package:

```
CREATE OR REPLACE PACKAGE pck_item AS
   FUNCTION chk_item(p_itemno IN NUMBER)
   RETURN NUMBER;
   PROCEDURE proc_item(
      p_itemno IN NUMBER,
      p_qty    IN NUMBER
   );
END pck_item;
/
SQL> @ "C:\Users\Desktop\package.sql"
Package created
```

Creating Package body:

```
CREATE OR REPLACE PACKAGE BODY pck_item AS
   FUNCTION chk_item(p_itemno IN NUMBER)
   RETURN NUMBER
   IS
      v_count NUMBER;
   BEGIN
      SELECT COUNT(*)
      INTO v_count
      FROM item_master
      WHERE itemno = p_itemno;
      IF v_count = 1 THEN
         DBMS_OUTPUT.PUT_LINE('Item exists.');
         RETURN 1;
      ELSE
         DBMS_OUTPUT.PUT_LINE('Item not found.');
         RETURN 0;
      END IF;
   END chk_item;
```

```
PROCEDURE proc_item(
   p_itemno IN NUMBER,
   p_qty    IN NUMBER
)
IS
   v_stock item_master.stock%TYPE;
   v_name  item_master.name%TYPE;
   v_price item_master.unit_price%TYPE;
   v_total NUMBER;
   v_result NUMBER;
BEGIN
   v_result := chk_item(p_itemno);
   IF v_result = 0 THEN
      RETURN;
   END IF;
   SELECT stock, name, unit_price
   INTO v_stock, v_name, v_price
   FROM item_master
   WHERE itemno = p_itemno;
   IF p_qty > v_stock THEN
      DBMS_OUTPUT.PUT_LINE( 'Insufficient stock! Available stock: ' || v_stock );
   ELSE
      v_total := v_price * p_qty;
        INSERT INTO item_trans
      VALUES (p_itemno, p_qty, SYSDATE);
       UPDATE item_master
      SET stock = stock - p_qty
      WHERE itemno = p_itemno;
      DBMS_OUTPUT.PUT_LINE('Item       : ' || v_name);
      DBMS_OUTPUT.PUT_LINE('Quantity   : ' || p_qty);
      DBMS_OUTPUT.PUT_LINE('Unit Price : ' || v_price);
      DBMS_OUTPUT.PUT_LINE('Total      : ' || v_total);
```

```
            COMMIT;
        END IF;
    EXCEPTION
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
    END proc_item;
END pck_item;
/
```

SQL> @ "C:\Users\Desktop\pck_item.sql"

Package body created**.**

SQL> set serveroutput on;

PL/SQL Code:

```
DECLARE
    v_itemno NUMBER := &accept_itemno;
    v_qty    NUMBER := &quantity;
BEGIN
    pck_item.proc_item(v_itemno, v_qty);
END;
/
```

SQL> @ "C:\Users\Desktop\item.sql"

```
Enter value for accept_itemno: 1
old   2:     v_itemno NUMBER := &accept_itemno;
new   2:     v_itemno NUMBER := 1;
Enter value for quantity: 5
old   3:     v_qty    NUMBER := &quantity;
new   3:     v_qty    NUMBER := 5;
Item exists.
Item       : Mouse
Quantity   : 5
Unit Price : 15
Total      : 75

PL/SQL procedure successfully completed.
```

SQL> @ "C:\Users\Desktop\item.sql"

```
Enter value for accept_itemno: 2
old   2:     v_itemno NUMBER := &accept_itemno;
new   2:     v_itemno NUMBER := 2;
Enter value for quantity: 20
old   3:     v_qty    NUMBER := &quantity;
new   3:     v_qty    NUMBER := 20;
Item exists.
Insufficient stock! Available stock: 15

PL/SQL procedure successfully completed.
```

SQL> @ "C:\Users\Desktop\item.sql"

```
Enter value for accept_itemno: 9
old   2:     v_itemno NUMBER := &accept_itemno;
new   2:     v_itemno NUMBER := 9;
Enter value for quantity: 5
old   3:     v_qty    NUMBER := &quantity;
new   3:     v_qty    NUMBER := 5;
Item not found.

PL/SQL procedure successfully completed.
```

PROGRAM 6.

Create a package which includes a function to compute the factorial of a number, a procedure to compute the value of nCr, and another procedure to compute nPr both uses the factorial function. Execute the package program for the required calculation

Creating Package:

```
CREATE OR REPLACE PACKAGE math_package AS
  FUNCTION factorial(n NUMBER) RETURN NUMBER;
   PROCEDURE calc_nCr(n NUMBER, r NUMBER);
   PROCEDURE calc_nPr(n NUMBER, r NUMBER);
END math_package;
/
SQL> @ "C:\Users\Desktop\package.sql"
Package created.
```

Creating Package body:

```
CREATE OR REPLACE PACKAGE BODY math_package AS
   FUNCTION factorial(n NUMBER)
   RETURN NUMBER
   IS
      f NUMBER := 1;
   BEGIN
      IF n = 0 THEN
         RETURN 1;
      ELSE
         FOR i IN 1..n LOOP
            f := f * i;
         END LOOP;
         RETURN f;
      END IF;
   END factorial;
   PROCEDURE calc_nCr(n NUMBER, r NUMBER)
   IS
      result NUMBER;
```

```
    BEGIN
        result := factorial(n) /
                (factorial(r) * factorial(n - r));
        DBMS_OUTPUT.PUT_LINE(
            'nCr (' || n || ', ' || r || ') = ' || result
        );
    END calc_nCr;
    PROCEDURE calc_nPr(n NUMBER, r NUMBER)
    IS
        result NUMBER;
    BEGIN
        result := factorial(n) / factorial(n - r);
        DBMS_OUTPUT.PUT_LINE(
            'nPr (' || n || ', ' || r || ') = ' || result
        );
    END calc_nPr;
END math_package;
/
SET SERVEROUTPUT ON;
SQL> @ "C:\Users\Desktop\math_package.sql"
Package body created.
```

PL/SQL Code:

```
DECLARE
    n NUMBER;
    r NUMBER;
BEGIN
    n := &n;
    r := &r;
    math_package.calc_nCr(n, r);
    math_package.calc_nPr(n, r);
END;
/
```

SQL> @ "C:\Users\Desktop\NCR.sql"

```
SQL> @"D:\DBMS\NCR.sql"
Enter value for n: 5
old   6:     n := &n;
new   6:     n := 5;
Enter value for r: 2
old   7:     r := &r;
new   7:     r := 2;
nCr (5, 2) = 10
nPr (5, 2) = 20

PL/SQL procedure successfully completed.
```

PROGRAM 7.

Create a trigger to update the MASTER table when a record is inserted into SALES table and create another trigger to update the MASTER table when a record is inserted or updated or deleted in NEWSTOCK table. Assume the suitable columns for all the tables.

Create your own user:

```
CREATE USER student IDENTIFIED BY stud123;

GRANT CONNECT, RESOURCE TO student;

CONNECT student/stud123;
```

Creating tables:

```
SQL> CREATE TABLE master
(
   product_id NUMBER PRIMARY KEY,
   product_name VARCHAR2(100),
   total_sales NUMBER DEFAULT 0,
   current_stock NUMBER DEFAULT 0
);
Table created.

SQL> CREATE TABLE sales
(
   sale_id NUMBER PRIMARY KEY,
   product_id NUMBER,
   quantity_sold NUMBER,
   sales_date DATE,
   CONSTRAINT fk_sales_product
   FOREIGN KEY (product_id)
   REFERENCES master(product_id)
);
Table created.
```

```
SQL> CREATE TABLE newstock
(
   stock_id NUMBER PRIMARY KEY,
   product_id NUMBER,
   quantity_added NUMBER,
   stock_date DATE,
   CONSTRAINT fk_newstock_product
   FOREIGN KEY (product_id)
   REFERENCES master(product_id)
);
```

Table created.

Inserting values:

```
SQL> INSERT INTO master VALUES (1, 'Laptop', 0, 100);
```

1 row created.

```
SQL> INSERT INTO master VALUES (2, 'Smartphone', 0, 200);
```

1 row created.

```
SQL> INSERT INTO master VALUES (3, 'Table', 0, 150);
```

1 row created.

```
SQL> COMMIT;
```

Commit complete.

```
SQL> SELECT * FROM master;
```

PRODUCT_ID	PRODUCT_NAME	TOTAL_SALES	CURRENT_STOCK
1	Laptop	0	100
2	Smartphone	0	200
3	Table	0	150

Creating trigger 1 – update master after inserting into sales

```
create or replace trigger trg_after_insert_sales
after insert on sales
for each row begin
```

```
update master
set total_sales = nvl(total_sales, 0) + :new.quantity_sold,
current_stock = nvl(current_stock, 0) - :new.quantity_sold
where product_id = :new.product_id;
end;
/
SQL> @ "C:\Users\Desktop\Trigger1.sql"
Trigger created.
```

Creating trigger 2– update master on insert / update / delete in newstock

```
create or replace trigger trg_dml_newstock after insert or update or delete on newstock for each row
begin
if inserting then
update master
set current_stock = nvl(current_stock, 0) + :new.quantity_added where
product_id = :new.product_id;
elsif deleting then
update master
set current_stock = nvl(current_stock, 0) - :old.quantity_added
where product_id = :old.product_id;
elsif updating then
update master
set current_stock = current_stock - :old.quantity_added + :new.quantity_added
where product_id = :new.product_id;
end if;
end;
/
SQL> @ "C:\Users\Desktop\Trigger2.sql"
Trigger created.
```

Testing the triggers Inserting sales records :

```
SQL>insert into sales values (101, 1, 5, sysdate);
```

1 row created.

SQL>insert into sales values (102, 2, 10, sysdate);

1 row created. SQL>select *from sales;

SALE_ID	PRODUCT_ID	QUANTITY_SOLD	SALES_DAT
101	1	5	12-NOV-25
102	2	10	12-NOV-25

Inserting new stock records

SQL> insert into newstock values (201, 1, 20, sysdate);

1 row created.

SQL>insert into newstock values (202, 3, 15, sysdate);

1 row created.

SQL>commit;

Commit complete.

SQL>select *from master;

PRODUCT_ID	PRODUCT_NAME	TOTAL_SALES	CURRENT_STOCK
1	laptop	5	115
2	smartphone	10	190
3	table	0	165

Updating and deleting stock:

SQL>update newstock set quantity_added = 50 where stock_id = 201;

1 row updated.

SQL>delete from newstock where stock_id = 202;

1 row deleted.

SQL>select * from master;

PRODUCT_ID	PRODUCT_NAME	TOTAL_SALES	CURRENT_STOCK
2	Smartphone	10	190
1	Laptop	5	145
3	Table	0	150

SQL>select * from sales;

SALE_ID	PRODUCT_ID	QUANTITY_SOLD	SALES_DAT
101	1	5	12-NOV-25
102	2	10	12-NOV-25

SQL>select * from newstock;

STOCK_ID	PRODUCT_ID	QUANTITY_ADDED	STOCK_DAT
201	1	50	12-NOV-25

SQL>select * from sales;

SALE_ID	PRODUCT_ID	QUANTITY_SOLD	SALES_DAT
101	1	5	12-NOV-25
102	2	10	12-NOV-25

SQL>select * from newstock;

STOCK_ID	PRODUCT_ID	QUANTITY_ADDED	STOCK_DAT
201	1	50	12-NOV-25